

Student Guide — Lab 5: pavo Walkthrough

Quantifying reef-fish color patterns · EEB 187 Week 6

EEB 187 — Ecology & Evolution of Color

May 6, 2026

Table of contents

How to use this guide	2
1. Orient yourself in RStudio	2
Find the walkthrough script	3
2. Get your images in place	4
Demo fish (already loaded — nothing to do)	4
Your lineage's fish (Part B)	4
Option A — Use the pre-loaded library (zero work)	4
Option B — Bring your own JPGs	4
3. Step 0 — Load packages and helpers	5
4. Step 1 — Load the three demo fish	6
5. Step 2 — Classify each image into k color classes	7
6. Step 3 — Adjacency analysis (the heart of the lab)	8
7. Step 4 — Extract the headline metrics	10
8. Step 5 — Visualize the comparison	11
9. Part B — Your three fish	11
Step B.1 — Pick three species	11
Step B.2 — Segment your own fish	11
Path 1 — Local with <code>segment_fish()</code>	12
Path 2 — Web tool	12
Path 3 — Manual crop	12
Step B.3 — Run the same pipeline	12
10. Step C — Compare your fish to the demo	13
11. Step D — Biological interpretation	13

12. Submit	14
Common gotchas (read this if you're stuck)	14
What's next	15

How to use this guide

This is the **step-by-step companion** to the live demo. Use it when:

- You **fell behind** in lab and need to catch up
- You want to **try the analysis at home** before lab on a different fish
- You're **doing Part B** (your three lineage fish) and want a written reference for each step

It pairs with three other files in your lab repo:

File	What it's for
scripts/walkthrough.R	The R script you'll actually run line-by-line
scripts/helpers.R	Helper functions (segment_fish, simple_coldists, etc.) — you load these once at session start
lab-05-pavo-intro.qmd	The worksheet where you fill in your answers and submit

Read this guide top-to-bottom the first time. Once you've done it once, you can use it as a quick lookup.

Lost? Stuck?

Every step here ends with a “**What to do if it didn't work**” box. Read those before posting in the channel — most fixes are one-line.

1. Orient yourself in RStudio

When RStudio opens (whether locally, via Docker, Binder, or Codespaces), it has four panes:

TOP-LEFT: Source pane
(Where walkthrough.R will
open after you click it)

TOP-RIGHT: Environment
(Variables you've created;
empty when you start)

BOTTOM-LEFT: Console
(Where you type R commands
and see the output)

BOTTOM-RIGHT: Files /
Plots / Help / Packages
(Click "Files" to navigate)

Your starting working directory depends on how you launched RStudio:

- **Docker:** opens in `~/lab` automatically (the lab repo is mounted there)
- **Native:** opens wherever you last left it. Click **Session** → **Set Working Directory** → **Choose Directory**, navigate to `eeb187-pavo-lab`, click *Open*.
- **Binder / Codespaces:** opens in the lab repo by default

Confirm where R thinks you are — in the Console (bottom-left), type:

```
getwd()
```

You should see `/home/rstudio/lab` (Docker / Binder / Codespaces) or `/Users/yourname/eeb187-pavo-lab` (Native). If it's something else, you need to fix it before continuing — see the box below.

⚠ If `getwd()` doesn't show the lab repo

- **Docker:** stop the container and restart from inside the cloned repo. Run `cd ~/eeb187-pavo-lab` *before* running `docker run`.
- **Native:** in RStudio, **Session** → **Set Working Directory** → **To Project Directory** if you opened the `.Rproj` file. Otherwise navigate manually as above.
- **Binder:** this should always work — refresh the browser tab.

Find the walkthrough script

Open the **Files pane** (bottom-right). You should see (among others):

- `scripts/` — folder. Click to expand.
 - `walkthrough.R` ← **this is the live-coded script**
 - `helpers.R` — the helper functions
 - `verify_pipeline.R` — runs the full pipeline as a smoke test
- `images/` — folder. Has subfolders `demo/` (the 3 demo fish, pre-loaded) and `student-fish/` (your assigned lineage's 5 starter species).
- `lab-05-pavo-intro.qmd` — the worksheet you'll submit
- `README.md`, `prep.html`, etc. — supporting docs

Click `walkthrough.R` in the Files pane. It opens in the Source pane (top-left).

2. Get your images in place

The lab uses two sets of images:

Demo fish (already loaded — nothing to do)

Three demo fish — sergeant major, cleaner wrasse, mandarin fish — live at `images/demo/segmented/`. They're pre-segmented (background removed, white-padded, standardized canvas). You'll use these in **Part A** of the worksheet.

Your lineage's fish (Part B)

You have two options:

Option A — Use the pre-loaded library (zero work)

Each lineage already has 5 starter species in `images/student-fish/<lineage>/`. To see what's in yours, in the Console:

```
list.files("images/student-fish/chaetodontidae")    # change lineage as needed
```

Pick three that span pattern types — one with vertical bars, one with horizontal stripe, one spotty/reticulate.

Option B — Bring your own JPGs

Save raw fish photos with **lowercase, hyphenated** filenames (e.g., `pomacanthus-imperator.jpg`, NOT `Pomacanthus Imperator.JPG` or anything with spaces).

Where to put them depends on your launch path:

Launch	Where
Native	drop into your local <code>eeb187-pavo-lab/images/student-fish/<lineage>/</code> folder — no upload needed
Docker	drop into the same folder on your host (the bind mount makes them appear at <code>~/lab/images/student-fish/<lineage>/</code> inside the container automatically)
Binder	RStudio's Files pane (bottom-right) → navigate to <code>images/student-fish/<lineage>/</code> → click Upload → drop your JPGs
Codespaces	drag-drop into the VS Code file explorer at <code>images/student-fish/<lineage>/</code>

If you have raw photos that still have water/reef in the background, segment them first — see Section 8.

3. Step 0 — Load packages and helpers

This is the **most-skipped step** that causes 80% of student errors. Run it first, every time.

⚠ ALL FOUR lines must run after every session start

If you restart R (e.g., **Session** → **Restart R**, or Docker container restart, or you reload Binder), the entire global environment is wiped — every package, every variable, every helper. You must re-run Step 0 in full. Running just *one* of the four lines is the most common reason students get “could not find function” errors mid-pipeline:

Line you skipped	Error you'll get
<code>library(pavo)</code>	Error: could not find function "getimg" (also classify, adjacent)
<code>library(magick)</code>	unable to load shared object ... magick.so (or some plotting errors)
<code>library(tidyverse)</code>	Error: could not find function "bind_rows" (or pivot_longer, ggplot)
<code>source("scripts/helpers.R")</code>	Error: could not find function "simple_coldists" (also pick_white_bg, segment_fish, composite_to_white)

In the Console (or by clicking through the script in the Source pane):

```
library(pavo)           # loads getimg(), classify(), adjacent()
library(magick)          # image I/O backend pavo uses
library(tidyverse)       # bind_rows, pivot_longer, ggplot
source("scripts/helpers.R") # segment_fish, simple_coldists, pick_white_bg, ...
packageVersion("pavo")
```

What you should see:

```
Linking to ImageMagick 6.9.12.98 ...
Attaching core tidyverse packages ...
[1] '2.9.0'
```

The four `library` / `source` lines above all need to run cleanly. The `source("scripts/helpers.R")` line is silent (no output unless something's wrong) — but it's loading 6 helper functions: `segment_fish`, `composite_to_white`, `flip_to_left_lateral`, `simple_coldists`, `pick_white_bg`, `pick_col`. Skip it and Step 3 of the analysis will fail.

💡 Pro tip

At the top of `walkthrough.R` you'll see these four lines as block comments. Highlight all four lines in the Source pane (or the whole Step 0 block) and press **Cmd-Enter** (Mac) / **Ctrl-Enter** (Windows / Linux) to send them all to the Console at once.

⚠️ If Step 0 didn't work

Error	Fix
there is no package called 'pavo'	The package didn't install. Run <code>install.packages("pavo")</code> and watch the output. If it errors, post in the channel.
unable to load shared object ... magick.so	macOS native: <code>brew install imagemagick</code> , then re-run <code>install.packages("magick")</code> .
cannot open file 'scripts/helpers.R'	Your working directory isn't the lab repo. Run <code>getwd()</code> and fix per Section 1.
[1] '2.8.0' (or older)	Update pavo: <code>install.packages("pavo")</code> .

4. Step 1 — Load the three demo fish

The 3 demo fish are in `images/demo/segmented/`. In the Console:

```
sergeant_major <- getimg("images/demo/segmented/sergeant-major.jpg")
cleaner_wrasse <- getimg("images/demo/segmented/cleaner-wrasse.jpg")
mandarin_fish  <- getimg("images/demo/segmented/mandarin-fish.jpg")
```

`getimg()` reads each JPG into pavo's `ring` object — a 3D array (`height × width × {R, G, B}`). **You should not see any output** — silent success means it worked.

Confirm by inspecting one:

```
class(sergeant_major)      # "ring" "array"
dim(sergeant_major)        # height × width × 3
attr(sergeant_major, "imgname")
```

Plot all three side-by-side:

```
par(mfrow = c(1, 3))
plot(sergeant_major); title("sergeant major")
plot(cleaner_wrasse); title("cleaner wrasse")
plot(mandarin_fish); title("mandarin fish")
par(mfrow = c(1, 1))
```

The plots appear in the **Plots tab** (bottom-right pane, below the Files tab).

💡 Predict before you measure

Before you compute anything, write down which fish you think will have:

- the **highest A** (most vertical-edge dominance)
- the **lowest A** (most horizontal-edge dominance)
- A near 1 (no directional bias)

Hint: $A = \text{vertical edges} \div \text{horizontal edges}$. Bars run vertically (so they CREATE vertical edges where they meet the body color); stripes run horizontally.

⚠️ If Step 1 didn't work

Error	Fix
cannot open the connection	Working directory wrong. <code>getwd()</code> should print the lab repo.
Error: object 'getimg' not found Plot is empty / weird	You skipped <code>library(pavo)</code> . Back to Step 0. The image file is corrupted. Re-clone the repo.

5. Step 2 — Classify each image into k color classes

`classify(img, kcols = k)` runs **k-means on every pixel's RGB triplet**. Each pixel gets snapped to one of k color centroids; the result is a “classified image” where the pattern is reduced to k discrete regions.

Choose k based on visible color count + 1 (the +1 is for the white background that segmentation added):

```
sm_class <- classify(sergeant_major, kcols = 4) # yellow, black, pale belly, WHITE bg
cw_class <- classify(cleaner_wrasse, kcols = 4) # dark stripe, mid blue, light, WHITE bg
mf_class <- classify(mandarin_fish, kcols = 6)  # 5 fish colors + WHITE bg
```

You'll see Image classification in progress... while each runs (~5 seconds each).

Inspect the palettes:

```
attr(sm_class, "classRGB") # 4 rows of (R, G, B) values, one per class
```

One row will have R G B 1 — that's the **white background** class. Note its row number (1, 2, 3, or 4) — we'll need it in Step 3.

Plot before / after for one fish:

```
par(mfrow = c(2, 1))
plot(sergeant_major); title("Sergeant major - original")
plot(sm_class);      title("Sergeant major - classified, k = 4")
par(mfrow = c(1, 1))
```

You should see the original photo replaced by 4 flat-color regions. The black bars should still be visible as a discrete class; the white background should be a uniform region.

💡 Why k matters

Try `classify(sergeant_major, kcols = 2)` and replot. The yellow body and black bars **collapse into one class** because $k=2$ forces a binary split (fish vs. background). This destroys the pattern. **Choosing k too low is the most common student error in Part B.**

Rule of thumb: count the visibly-distinct colors on the fish, add 1 for the white background.

⚠️ If Step 2 didn't work

Error	Fix
Error: 'kcols' missing	Typo — it's <code>kcols</code> (with the 's'), not <code>kcol</code> .
Classified image looks washed-out	k is too small. Try $k+1$.
Two near-identical rows in <code>classRGB</code>	k is too high. Try $k-1$.

6. Step 3 — Adjacency analysis (the heart of the lab)

This step **drops a regular grid** over each classified image, **counts the transitions** between color classes between adjacent grid cells, and **computes the seven Endler 2012 pattern metrics**.

For each fish you need three things first:

1. The pairwise **color distances** (so `adjacent()` can compute boundary strength metrics)
2. The **white background class index** (so `adjacent()` can exclude it)
3. The actual `adjacent()` call

```
# Sergeant major
sm_cd  <- simple_coldists(sm_class)
sm_bkg <- pick_white_bg(sm_class)
sm_adj <- adjacent(sm_class, xpts = 200, xscale = 5,
```



```

        coldists = sm_cd, bkgID = sm_bkg, exclude = "background")

# Cleaner wrasse
cw_cd <- simple_coldists(cw_class)
cw_bkg <- pick_white_bg(cw_class)
cw_adj <- adjacent(cw_class, xpts = 200, xscale = 5,
                  coldists = cw_cd, bkgID = cw_bkg, exclude = "background")

# Mandarin fish
mf_cd <- simple_coldists(mf_class)
mf_bkg <- pick_white_bg(mf_class)
mf_adj <- adjacent(mf_class, xpts = 200, xscale = 5,
                  coldists = mf_cd, bkgID = mf_bkg, exclude = "background")

```

`xpts = 200` means a 200×200 sample grid. `xscale = 5` is the body length in cm — only matters for distance metrics.

Each `adjacent()` call returns a one-row data frame with ~30 columns of metrics. Stack them:

```

fish_adj <- bind_rows(
  sergeant_major = sm_adj,
  cleaner_wrasse = cw_adj,
  mandarin_fish = mf_adj,
  .id = "fish"
)

```

⚠ If Step 3 didn't work

Error	Fix
could not find function "simple_coldists"	You skipped <code>source("scripts/helpers.R")</code> in Step 0. Run it now.
<code>m_dS</code> and <code>m_dL</code> come back as NA	<code>coldists</code> not passed. Re-check the <code>coldists = sm_cd</code> argument.
All metrics look weird, A way off	<code>bkgID</code> set to wrong class. Confirm <code>attr(sm_class, "classRGB")[sm_bkg,]</code> is roughly (1, 1, 1).
Slow (~5–10 sec per fish)	Normal. <code>xpts=200</code> is the speed/accuracy sweet spot.

7. Step 4 — Extract the headline metrics

pavo's `adjacent()` returns ~30 columns. We focus on seven from Endler (2012):

Metric	Meaning
k	number of color classes
m	transition density — how <i>busy</i> the pattern is per unit area
A	vertical edges $\div$ horizontal edges — the headline metric
Jc	color heterogeneity — how <i>evenly</i> colors are used
Jt	transition heterogeneity — how diverse the boundaries are
m_dS	mean chromatic boundary strength — how saturated edges are
m_dL	mean luminance boundary strength — how bright/dark edges are

```
metrics <- tibble(
  fish    = c("sergeant_major", "cleaner_wrasse", "mandarin_fish"),
  family  = c("Pomacentridae", "Labridae", "Callionymidae"),
  k       = pick_col(fish_adj, c("k", "kcols")),
  m       = pick_col(fish_adj, c("m", "m_r")),
  A       = pick_col(fish_adj, c("A", "Asp")),
  Jc      = pick_col(fish_adj, c("Jc")),
  Jt      = pick_col(fish_adj, c("Jt")),
  m_dS    = pick_col(fish_adj, c("m_dS", "Cs_dS")),
  m_dL    = pick_col(fish_adj, c("m_dL", "Cs_dL")),
)
print(metrics)
```

`pick_col()` accepts a vector of candidate names and returns the first one that exists — pavo's column names vary slightly between releases, this makes the code robust.

You should see something like:

```
# A tibble: 3 × 9
  fish      family      k      m      A      Jc      Jt      m_dS      m_dL
  <chr>      <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 sergeant_major Pomacentridae  4 0.42  1.54  0.71 ...    ...    ...
2 cleaner_wrasse Labridae      4 0.31  0.60  0.62 ...    ...    ...
3 mandarin_fish  Callionymidae  6 0.55  1.02  0.81 ...    ...    ...
```

Did your prediction hold? $A > 1$ for the bars, $A < 1$ for the stripe, $A = 1$ for the reticulate.

Record these values in Part A's metric table on the worksheet (`lab-05-pavo-intro.qmd`).

8. Step 5 — Visualize the comparison

```
metrics %>%
  pivot_longer(c(m, A, Jc, m_dL), names_to = "metric", values_to = "value") %>%
  ggplot(aes(x = fish, y = value, fill = family)) +
    geom_col() +
    facet_wrap(~metric, scales = "free_y") +
    scale_fill_manual(values = c("Pomacentridae" = "#5BC0EB",
                                "Labridae"       = "#3FB8AF",
                                "Callionymidae"  = "#FFC946")) +
    labs(title = "Pattern metrics across three reef families",
         x = NULL, y = "metric value") +
    theme_minimal(base_size = 14) +
    theme(legend.position = "bottom",
          axis.text.x = element_text(angle = 30, hjust = 1))
```

Four panels, three fish each. The **A** panel is the punchline — bars highest, stripe lowest, reticulate in the middle.

This completes **Part A** of the worksheet.

9. Part B — Your three fish

Now you do the same on three species from your assigned lineage. Same pipeline; just swap your image paths in.

Step B.1 — Pick three species

Browse `images/student-fish/<your-lineage>/` (already on your laptop) and pick:

- One with **strong vertical bars** → predict $A > 1$
- One with a **horizontal element** (stripe, band) → predict $A < 1$
- One that's **spotted, reticulate, or solid** → predict $A \approx 1$

Write your three picks in the worksheet's Part B Step 1.

Step B.2 — Segment your own fish

If your three picks are from the pre-loaded `student-fish/<lineage>/` library, **they're already cropped** and you can skip ahead to Step B.3 with `getimg()` directly.

If you brought a raw photo with water/reef in the background, segment it first — pick the path that works for your install:

Path 1 — Local with `segment_fish()`

```
seg_path <- segment_fish("images/student-fish/<lineage>/your-fish.jpg")
my_fish  <- getimg(seg_path)
```

Path 2 — Web tool

Drag-drop your fish JPG into [remove.bg](#), [photoroom.com](#), or [clipdrop.co/remove-background](#). Save the transparent PNG into the same folder. Then:

```
seg_path <- composite_to_white("images/student-fish/<lineage>/your-fish.png")
my_fish  <- getimg(seg_path)
```

Path 3 — Manual crop

Open in Preview.app (macOS) or Photos (Windows). Drag a tight rectangle around the fish, **Tools** → **Crop**, save. Then load directly with `getimg(...)` — skip the segment helpers.

Step B.3 — Run the same pipeline

```
my_fish <- getimg(seg_path)          # path from Step B.2
plot(my_fish)                       # confirm head on LEFT (left-lateral)
                                     # if not: flip_to_left_lateral(seg_path); my_fish

my_k    <- 4                         # = visible fish colors + 1 (for white bg)
my_class <- classify(my_fish, kcols = my_k)
plot(my_class)                      # does this match the original pattern?

my_cd    <- simple_coldists(my_class)
my_bkg   <- pick_white_bg(my_class)
my_adj   <- adjacent(my_class, xpts = 200, xscale = 5,
                     coldists = my_cd, bkgID = my_bkg, exclude = "background")

my_metrics <- tibble(
  fish = "your_species_name",
  k     = pick_col(my_adj, c("k", "kcols")),
  m     = pick_col(my_adj, c("m", "m_r")),
  A     = pick_col(my_adj, c("A", "Asp")),
  Jc    = pick_col(my_adj, c("Jc")),
  Jt    = pick_col(my_adj, c("Jt")),
  m_dS  = pick_col(my_adj, c("m_dS", "Cs_dS")),
  m_dL  = pick_col(my_adj, c("m_dL", "Cs_dL")),
```

```
)  
print(my_metrics)
```

Do this **three times** (once per fish), record the values in the worksheet's Part B metric table.

 Manual-crop fallback uses different code

If you used **Path 3 (manual crop)** in Step B.2, your image doesn't have a white background class. **Drop the `bkgID = my_bkg`, `exclude = "background"` arguments from `adjacent()`:**

```
my_adj <- adjacent(my_class, xpts = 200, xscale = 5, coldists = my_cd)
```

Your A values will be slightly less dramatic than the demo (closer to 1) because background pixels pollute the analysis. Note this in your worksheet — the rubric accommodates it.

10. Step C — Compare your fish to the demo

Look at your A values vs. the demo fish A values:

Demo fish	Family	Pattern	Demo A
Sergeant major	Pomacentridae	Vertical bars	~1.5
Cleaner wrasse	Labridae	Horizontal stripe	~0.6
Mandarin fish	Callionymidae	Reticulate	~1.0

For each of your three fish, ask:

1. Did the A statistic correctly classify it by pattern orientation? ($A > 1$ for your bar fish, $A < 1$ for your stripe fish, $A \approx 1$ for your reticulate fish)
2. Which demo fish does each of yours most resemble in metric space?

Answer these in the worksheet's **Part B Step 4**.

11. Step D — Biological interpretation

Pick **one** of your three fish and propose a **selection pressure** that might have favored its pattern. The worksheet gives you a menu:

- crypsis (camouflage)

- aposematism (warning coloration)
- species recognition
- sexual signaling
- predator deflection (eyespot)
- disruptive coloration
- “uncertain — here’s why”

The strongest answers **link your hypothesis back to specific measured metrics**. For example: “high `m_dS` suggests the saturation contrast at edges is large, which is consistent with aposematic warning rather than crypsis (which would predict low `m_dS`).”

Write 4–6 sentences in the worksheet’s **Part B Step 5**.

12. Submit

Render or export the worksheet (`lab-05-pavo-intro.qmd`) with your tables filled in:

- **HTML/PDF** — Click **Render** at the top of RStudio’s source pane
- **Or** copy the metric tables + your text answers into a Google Doc

Submit on BruinLearn by end of day.

Common gotchas (read this if you’re stuck)

Symptom	Likely cause	Fix
could not find function "getimg" (or classify, adjacent)	You didn’t <code>library(pavo)</code> after a session restart	Re-run Step 0 in full — all four lines
could not find function "simple_coldists" (or <code>pick_white_bg</code> , <code>pick_col</code> , <code>segment_fish</code>)	You didn’t <code>source("scripts/helpers.R")</code> at session start	Re-run Step 0 in full — all four lines
could not find function "bind_rows" (or <code>pivot_longer</code> , <code>ggplot</code>)	You didn’t <code>library(tidyverse)</code>	Re-run Step 0 in full — all four lines
cannot open the connection from <code>getimg()</code>	Working directory wrong	<code>getwd()</code> should print the lab repo path. Fix per Section 1.
Classified image is just two colors	k too small	Increase k by 1

Symptom	Likely cause	Fix
Classified palette has duplicate-looking rows	k too large	Decrease k by 1
A for vertical-bar fish came back < 1	Image is right-lateral (head on right), or k chose the wrong palette	(a) <code>flip_to_left_lateral(seg_path)</code> and re-load; (b) try a different k
All metrics are NA after <code>adjacent()</code>	Image is empty (all-white or all-black)	Re-load with <code>getimg()</code> and check <code>plot(img)</code> actually shows the fish
<code>bkgID</code> must be in <code>1:kcols</code>	<code>bkgID</code> from one fish was reused on another	Re-compute <code>pick_white_bg(...)</code> for each fish
Plots don't appear	Plot pane is closed or in a weird state	In RStudio: View → Panes → Console on Right to reset

What's next

There will be a **second pavo session** later in the term where we:

- Build a metric matrix across many species per lineage
- Cluster fish by pattern
- Compare clusters to phylogeny

That's the comparative-analysis follow-up. Today's lab gave you the tool; next time we use it for comparative biology.

Questions? Post in the EEB 187 channel. Most things you'll hit are documented above — search this guide first.